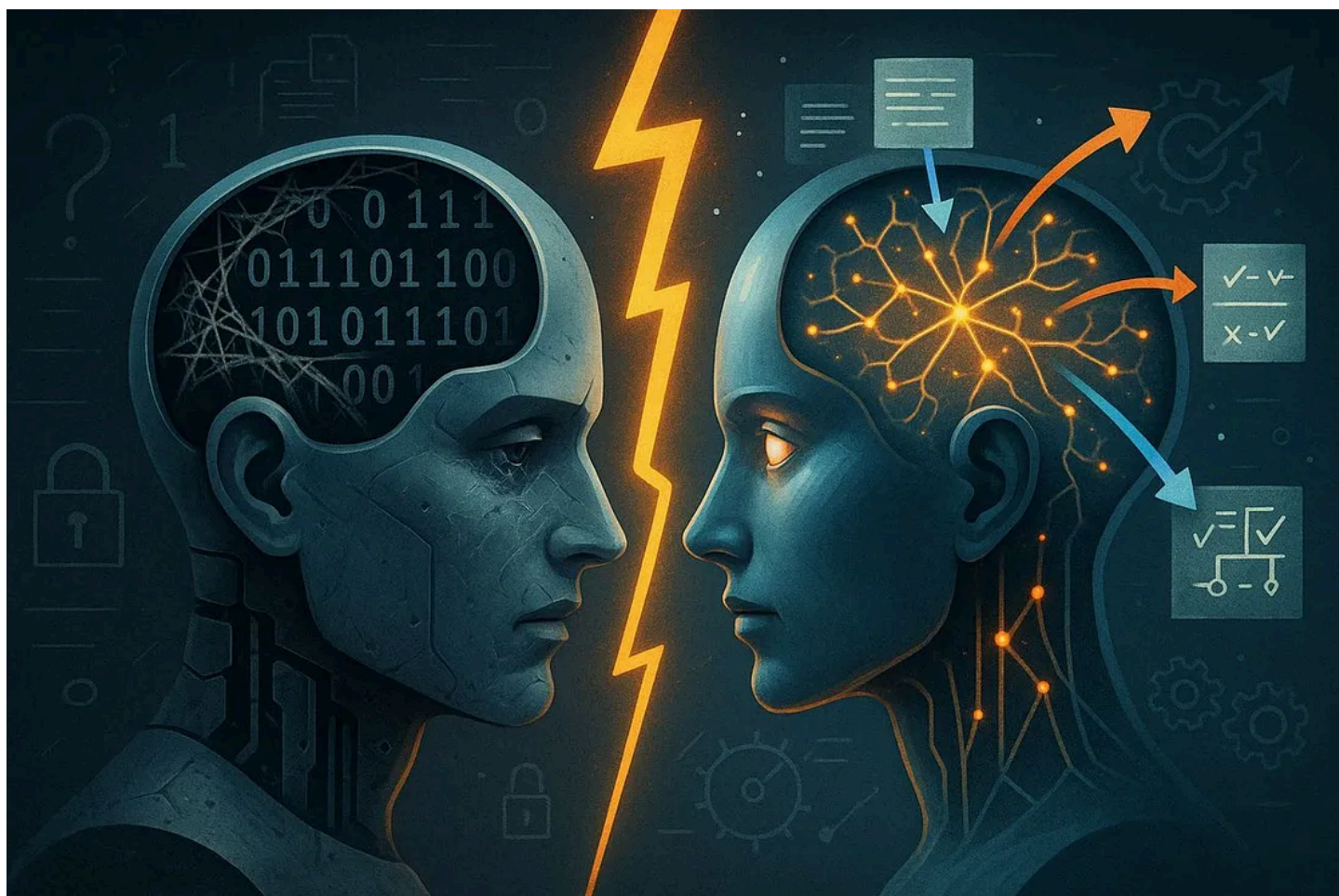


Has MIT Solved AI Adaptation?

Self-Adapting Language Models



Source: Author using GPT-4o

Recently, I've been covering several limitations 'frontier AI' models have to dial down the hype around 'as smart as a PhD' models (a barefaced lie).

And one thing AI models are embarrassingly bad at, or to be clearer, they just can't do it, is adapting to new data. This lack of adaptation is a death sentence for the model, preventing it from learning new skills on the fly.

Now, researchers at MIT think they have a potential solution they call *SEAL*, or **SE**lf-Adapting Language Models. The **excellent** results speak for themselves, with models achieving a performance increase in ARC, one of AI's most challenging benchmarks, of up to almost 80% (from 0%) with just this trick.

Join me to explore one of AI's glaring limitations, learning what makes training AIs different from how humans discover and learn, and understanding how a clever approach may have pulled us closer to models that finally do justice to the overhyped claims in AI.

Anthropomorphized Static Blobs

Currently, language models, which some incumbents claim are "*smart as PhDs*", are static, meaning they have a learning cutoff to which they stop learning.

But what does that mean?

They Don't Know What Happened Today.

To get the models you and me use, these first endure a long training regime, in which they first learn to imitate the world (via text) and then, in cases like reasoning models, they also undergo trial-and-error training, where models enter a ‘guess and verify’ loop until they figure how to solve problems and develop new skills (they do, as we saw last week in my newsletter).

And once the model is trained to our liking, it’s deployed. However, from then on, the model enters ‘inference mode’, **where it performs an infinite number of interactions with users, but will learn nothing**, and I mean nothing, from those interactions.

But don’t some of these products have ‘memory’ features, so they do remember past conversations?

Yes, but this feature is context-enhancing, not real learning.

If the model identifies something worth remembering, it adds it to ‘memory,’ which in reality means that piece of information will be added to the model’s prompt for every future interaction it has with you. But the model isn’t learning anything; it just has updated information for future interactions with you in the prompt; it’s like giving your boss a briefing of the situation before a meeting.

But why is a static model a problem?

Imagine your current self having to live the rest of their life without learning anything.

For instance, you might experience a friend betraying you tomorrow. Too bad, cause they will be able to betray you every single day of the rest of your life because you don’t internalize anything; **you’re a static blob of knowledge and experience from the past, unable to learn anything new.**

That’s the reality of our models.

The solution, of course, is to retrain them with new data, a term known as ‘fine-tuning.’

This is done periodically (for instance, OpenAI’s GPT-4o has changed multiple times in the past year, despite being the “same” model), **but this requires a significant amount of human data annotation** where members of the technical staff at OpenAI recall user feedback and data, and prepare it for the model for retraining.

Data annotation is, by far, one of the largest operational costs of big AI Labs, whether it’s internal or outsourced to companies like Scale AI or Surge AI.

Although this works, it has to be done all at once every several weeks or months. Instead, for more timely updates, the models rely on search engines like Perplexity, Google, or Bing to retrieve updated data in real-time.

However, search-engine retrieval is not the model learning anything new; it’s very similar to the memory feature we discussed earlier: simply adding a blob of context extracted from the search engine to the prompt and hoping the model utilizes it.

For example, if you open a conversation with ChatGPT to discuss the latest news on Iran, the next conversation you open with the model in another tab, that model will know nothing of what you last discussed with it in another window; it’s like talking to Dory from ‘Finding Nemo.’

The reason is that, in reality, the actual model remains unchanged, ever-ignorant of world changes, unable to adapt.

But what if there were a way to enable AIs to learn as they interact with you?

This, known as **test-time training**, is an idea that MIT may have finally streamlined.

The Advent of Test-Time Training

For centuries, **humans have relied chiefly on deductive reasoning.**

A genius amongst us observes something or has a unique idea, tests it, and propagates it as a new rule of nature. In other words, we can’t make infinite tests or process large amounts of data, so we rely on the unique intuition of some of us.

More formally, **deductive reasoning involves moving from the general to the specific**. We propose a new theory that is logically guaranteed to explain how the world works. It works, but, as mentioned, **requires genius-like behavior from the originator of the idea**.

With AI, **we have opened civilization to inductive reasoning at scale** (from the specific to the general). As these models can process vast amounts of data, we can make discoveries by observing patterns across massive datasets.

Let me give you an example. It took the genius of Isaac Newton to realize how gravity works. However, back in 2022, an AI ‘rediscovered’ the laws of gravity simply by observing interplanetary data; the model wasn’t even given insight into the fact that planets had mass, it inferred it by seeing patterns across a large set of data.

By the way, inductive learning is also how models like ChatGPT learn. By replicating a larger-than-life dataset, compression happens, meaning the model eventually finds the patterns that govern language (grammar, translation, etc); instead of the human method, teaching a language starting from the rules (grammar, notation, etc.) ChatGPT learns English by predicting how words follow each other.

However, test-time training, our topic of today, is neither of the two.

Test-Time Training, or TTT, **is the concept of training a model while it’s making predictions, allowing it to improve at the task it’s facing**. In other words, it runs into a task it does not know how to solve, figures it out, and internalizes the learning, directly updating the weights.

The reason TTT is not inductive nor deductive is because the goal isn’t generalization, going from specific to general or vice versa. Instead, the goal is to make the model better at that particular or highly similar tasks.

This introduces a “new” form of discovery, **transductive learning** (which humans also heavily leverage). Put another way, **it’s reasoning from specific examples to other specific examples without ending with a general rule**, just one that works for this case.

The point is that the model doesn’t need to update its entire understanding of the world every time it faces an unknown task; it just needs to learn how to do that task correctly.

Therefore, we hope that, through TTT, **AIs will learn on the fly**, adapting themselves to “known unknowns,” a critical component of human intelligence.

With adaptability, one could seriously start considering the possibility that we might end up creating real machine intelligence, because the efficient acquisition of new skills was — is — crucial to human intelligence.

However, test-time training is easier said than done, and more often than not, requires human intervention.

But, *what if we teach models to self-adapt?* Enter SEAL.

. . .

A Path to Self-Adaptation?

As mentioned, the idea of TTT is straightforward: upon encountering a new text passage, the model undergoes a self-adaptation process, **internalizing the latest data in real-time**.

But SEAL goes beyond that.

Instead, **the idea is to train the AI to improve this self-adaptation**. In other words, training the model to be a good trainer of itself; **the point isn’t only that the model learns new stuff on the fly, but that it has been literally trained to do so**. More formally, we are actively training an effective transductive learner.

SEAL involves two phases:

1. First, **we aim to train the best possible self-adapter**, one that generates data that facilitates effective learning.
2. After training, deploy the self-adapter in the wild and let it run wild and free, learning about the world.

Of course, the breakthrough comes in step 1, so let's explain that in detail.

The Genius Looped Trick

To understand SEAL, two key concepts must be briefly covered: RL and LoRA adapters.

- Reinforcement Learning (RL): Indirectly mentioned earlier, **it's the famous trial-and-error method**. Instead of training models to imitate data, we give them a goal, and the model will try and fail until it figures out how to reach the goal.
- **LoRA: The latter is a clever trick to fine-tune models without having to change the entire model**. The trick is to train small models and 'attach' them to the larger model, adapting their behavior accordingly. Think of this as if you could add a chip to your head that suddenly made you know something. If you detach the chip, you forget it. This is how Apple Intelligence works, by the way.

The reason we use LoRAs is that we don't actually want to train the base model until we verify the self-edit was good. As LoRAs enable us to train attachable surrogate models without modifying the base model, we can create a scalable process to teach a model to learn. This will make sense in a minute.

The SEAL process is as follows:

1. The model is shown a new text passage
2. It generates 'self-edits', transforming the text passage into 'learnable statements'
3. As we are conducting a trial-and-error exercise, the model explores different self-edit alternatives.
4. For every proposal it does, it trains a small adapter
5. We test every adapter, checking whether the new augmented model (base model + adapter) has learned the facts generated by that self-edit. This way, we can test if the self-edit worked without preemptively having to modify the base model
6. The 'self-edits' that have resulted in a well-calibrated adapter (trial and error) are then used to update the self-editing policy.

By 'self-editing policy' we mean the model.

[Source](#)

But what's the point?

The idea is that by doing this at scale, the model will improve over time in generating self-edits, thereby creating a good self-adapting model — **a model that can be deployed in the wild and is highly efficient at learning new information or skills**.

In the image below, you will appreciate how the model, after each round of improving its self-editing capabilities, becomes increasingly proficient at generating the trainable statements.

[Source](#)

Hence, once we have taught the model to be a good learner, **we can then trust it to deploy it in the wild and learn on the fly**.

But is the approach any good?

Results

The model was evaluated in two types of learning: **abstraction** and **knowledge**. In the first one, we are testing whether the model can solve complex pattern puzzles similar to those found in IQ tests. The second one tests the capacity to internalize new information.

- In **few-shot abstract reasoning**, using a subset of the ARC benchmark, SEAL learned to autonomously select the best data augmentations and training parameters for new tasks. **This resulted in a success rate of 72.5%, dramatically outperforming standard in-context learning (0%) and a baseline model using unoptimized self-edits (20%).**

Going from 0% to 72% without previous training on benchmark data is absolutely impressive.

*It's literally proof that the model can adapt to the task. And we aren't talking a powerful model; **researchers used Llama 3.2 1B, a very dumb model by modern standards.***

- In the domain of **knowledge incorporation**, SEAL was tasked with integrating factual information from text passages to answer questions without having access to the original text. **Fine-tuning on its self-generated data boosted question-answering accuracy to 47.0%**, which was superior to fine-tuning on the raw text alone (33.5%) and even surpassed the performance of using synthetic data generated by the larger GPT-4.1 model (46.3%), proving that SEAL training actually worked, as the model surpassed a smarter model at self-edits (in standard terms, GPT-4.1 is more capable, by far, than the model used here, Qwen2.5-7B).

In summary, SEAL works incredibly well in allowing models to acquire new skills or new facts.

. . .

Closing Thoughts

While most research focuses on making incremental improvements on what we have, other aims to open new paths of progress.

This study belongs in the latter group.

Some problems do remain, though.

- As with RL training pipeline, we still rely on verifiable rewards (in this case, the data we were training the model on had a clear correct answer), so this doesn't solve what's clearly the main unknown in AI today: **how to train models in a trial-and-error regime in areas where we don't know if the trial went well** (because then we can't measure the error, which is the learning signal).
- Another problem here is that we haven't yet fully figured out how to prevent **catastrophic forgetting**, which occurs when models forget older information when learning new material.

There are areas for improvement, but this research deflates bubbles not by releasing a slightly larger model with the same limitations as the previous ones, but by releasing research that tackles AI's real unsolved issues.

This is what real progress looks like.